

CSCI-GA.1170-003 Fundamental Algorithms

September 29, 2026

Problem Set 4

Lecturer: Julia Kempe

Due: 11:59pm on Monday, October 5

Problem 4-1 (Heaps: Beyond the Maximum)**15 points**

Recall that we defined a priority queue S together with the following operations (each of which runs in time $\log n$ except the second, which runs in time 1).

- $\text{INSERT}(S, x)$, which inserts x into S .
- $\text{MAXIMUM}(S)$, which returns the max element in S .
- $\text{EXTRACT-MAX}(S)$, which returns the max element and removes it from S .
- $\text{INCREASE-KEY}(S, i, x)$, which increases element i 's key to x .

For the purpose of this problem, we will call an algorithm *naive* if it only acts on S through these function calls.

Now assume the priority queue is implemented as a max-heap stored in an array $S[1 \dots n]$, and that you are also given access to the functions (the first four of which run in time 1 and the last in time $\log n$).

- $\text{PARENT}(i)$, $\text{LEFT}(i)$, $\text{RIGHT}(i)$, which return the parent, left child and right child of the i -th element.
- $\text{REMOVE}(S)$, which removes the rightmost leaf of S .
- $\text{MAX-HEAPIFY}(S, i)$, which lets the i -th element “float” down the heap.

For the purpose of this problem, we will call an algorithm *intelligent* if it additionally has access to these functions, and may also read and write any entry $S[i]$ directly in time 1. You may assume that all keys are distinct.

- (a) **(3 points)** Suppose you would like to find the third max in a heap (i.e., the third largest element of S). One naive approach might be to run the following code:

```

1 FIND-3RDMAX( $S$ )
2    $a \leftarrow \text{EXTRACT-MAX}(S)$ 
3    $b \leftarrow \text{EXTRACT-MAX}(S)$ 
4    $c \leftarrow \text{MAXIMUM}(S)$ 
5    $\text{INSERT}(S, a)$ 
6    $\text{INSERT}(S, b)$ 
7   return  $c$ 
```

However, this runs in time $1 + 4 \log n$. Your job is to find an “intelligent” solution which takes time close to 1. Give pseudocode and formally analyze the correctness and runtime of your algorithm.

- (b) **(4 points)** REPLACE-MAX(S, x) returns the max element of S and replaces it with x . Give a “naive” and an “intelligent” implementation, and analyze the running time of each as precisely as possible. How do they compare?
- (c) **(4 points)** REPORT-GREATER(S, v) returns all elements of S that are greater than a given value v , leaving S unchanged. Let m be the number of such elements. Give a “naive” implementation and analyze its running time in terms of m and n . Then give an “intelligent” implementation that runs in time $O(m + 1)$, independent of n , and argue its correctness.
Hint: If $S[i] \leq v$, what do you know about all the elements below $S[i]$ in the heap? Consider a recursive procedure that starts at the root.
- (d) **(4 points)** DECREASE-KEY(S, i, x) decreases element i ’s key to x . Give a “naive” and an “intelligent” implementation, and analyze both. (Each takes only a few lines of pseudocode calling the functions above.) Why can’t MAX-HEAPIFY be used to implement INCREASE-KEY in the same way?

Problem 4-2 (Quicksort vs Heapsort on Nice Arrays) 15(+3) points

We say that an array A is c -nice for a constant c if for all $1 \leq i < j \leq n$ such that $j - i \geq c$, we have that $A[i] \leq A[j]$. For example, a 1-nice array is completely sorted (in ascending order). In this problem we will sort such c -nice arrays A using various sorting algorithms and compare the results. (You may assume that A has n distinct elements.)

- (a) **(2 points)** For an element at position i in the array, what is the best lower bound you can give on its rank? What is the best upper bound?

Now consider a run of QUICKSORT on a c -nice array, using the PARTITION procedure from lecture, where the pivot element is chosen deterministically as the *first* element $A[p]$ of the subarray.

- (b) **(2 points)** Argue that after partitioning, the two subarrays $A[1 \dots q - 1]$ and $A[q + 1 \dots n]$ to the left and to the right of the pivot, respectively, are both c -nice.
- (c) **(3 points)** From the lecture you already know that the running time of quicksort on sorted arrays is $\Theta(n^2)$. Let $B(n)$ denote the best-case running time of QUICKSORT on a c -nice array with n elements. Using your results from (a) and (b), derive a recurrence for $B(n)$ and solve it.
- (d) **(5 points)** Recall that a min-heap is defined like a max-heap with all inequalities reversed; it supports INSERT and EXTRACT-MIN in time $O(\log m)$ on a heap with m elements, and can be built from m elements in time $O(m)$. Now use min-heaps (in a slightly clever way) to get an algorithm SORT-NICE(A, b) with the following specification. It takes as input an array A and an integer value $b \in \{1, \dots, n\}$, and always runs in time $O(n \log b)$. The output is always a permutation of the elements of A . However, if the input A is c -nice for some $c \leq b$, the output is the sorted version of A .
- (e) **(3 points)** When you are sorting arrays, you will not know what c is. Here is a general “guessing” strategy:

```

1 NICE-BUT-WEIRD-SORT( $A$ )
2   let  $i \leftarrow 0$ 
3   let  $b \leftarrow 2^i$ 
4    $B \leftarrow \text{SORT-NICE}(A, b)$ 
5   if  $B$  is not sorted, set  $i \leftarrow i + 1$ , goto line 3
6   return  $B$ 

```

Suppose the array is c^* -nice. Give the best runtime bound you can for this algorithm in terms of n and $\log c^*$.

- (f) **(Extra Credit)(3 points)** Change your choice for b to ensure that the algorithm above runs in time $O(n \log c^*)$.

Problem 4-3 (Assigning Letter Grades)

15 points

Hari Handiman and Mira Mekkanix are TAs for a course with n students, whose exam scores are stored in an array $A[1 \dots n]$ (you may assume all scores are distinct). The instructor has fixed $k - 1$ cutoff scores $c_1 < c_2 < \dots < c_{k-1}$, which split the scores into k letter-grade *bands*: a score s is in band j if $c_{j-1} < s \leq c_j$, where $c_0 = -\infty$ and $c_k = +\infty$. Hari and Mira must rearrange A so that all scores of band 1 come first, then all scores of band 2, and so on. The order of the scores *within* a band does not matter. They want to do this as fast as they can.

- (a) **(2 points)** Hari proposes simply sorting A using RANDOMIZED-QUICKSORT. Argue that this solves the problem in expected time $O(n \log n)$.

Mira notices that when $k = 2$ (pass/fail), a single partition suffices. Modify the lecture's PARTITION into a procedure PARTITION-VALUE(A, p, r, c) that rearranges $A[p \dots r]$ around a given *value* c (which need not appear in A) and returns an index q such that $A[p \dots q] \leq c < A[q + 1 \dots r]$. Its running time should be $O(r - p + 1)$.

- (b) **(2 points)** Mira generalizes her idea to k bands: first partition A around c_1 to move band 1 to the front, then partition the rest around c_2 , and so on. Argue that this is correct and runs in time $O(nk)$. Give an input on which it takes time $\Theta(nk)$.

Hari's algorithm is fast when k is large, and Mira's is fast when k is small. Hari and Mira join forces and design an algorithm, inspired by quicksort, that handles both cases gracefully. Instead of choosing a pivot among the scores, it partitions around one of the cutoffs:

1. If the current subarray is empty or there are no cutoffs left, return.
 2. Let c_m be the *middle* one of the remaining cutoffs $c_i, \dots, c_j$, i.e., $m = \lfloor (i + j)/2 \rfloor$.
 3. Call PARTITION-VALUE around c_m , and recurse on the left part with cutoffs $c_i, \dots, c_{m-1}$ and on the right part with cutoffs $c_{m+1}, \dots, c_j$.
- (c) **(5 points)** Write pseudocode BAND-SORT(A, p, r, i, j) for this algorithm, and state the initial call. Prove by induction that it correctly groups the scores by band.
- (d) **(4 points)** Show that the depth of the recursion is at most $\lceil \log_2 k \rceil$, and that the subarrays handled at any fixed depth are disjoint. Conclude that BAND-SORT runs in time $O(n \log k)$ in the worst case. What does this give for $k = 2$ and for $k = n$, and how does it compare with Hari's and Mira's algorithms?

- (e) **(2 points)** Suppose that in step 2 the algorithm instead picks the *smallest* remaining cutoff c_i . What is its worst-case running time, and which algorithm from earlier does it resemble? Relate this to the worst case of QUICKSORT from lecture.